

Elabora Documentos Técnicos Y De Usuario Del Software

GA10-220501097-AA10-EV01

Manual Técnico Y De Operación Del Sistema

Aprendiz:

Hernán Leonardo Quiroga Beltrán

Ficha:

3118323

Instructor:

Jairo Antonio Muñoz

Servicio Nacional de Aprendizaje – SENA

Tecnología en Análisis y Desarrollo de Software

2026

Tabla De Contenido

1. Introducción
2. Objetivos del Sistema
3. Descripción del Sistema de Información Desarrollado
4. Diseño Técnico del Sistema de Información
5. Despliegue y Configuración de Componentes
6. Resolución de Problemas
7. Referencias y Fuentes

1. Introducción

AutoReserve es una aplicación web de gestión de reservas de vehículos desarrollada como parte del programa de formación en Tecnología en Análisis y Desarrollo de Software del SENA. El sistema permite a usuarios anónimos explorar un catálogo de vehículos disponibles, mientras que los usuarios registrados pueden realizar reservas, marcar favoritos, consultar historial de reservas y dejar reseñas. Los administradores cuentan con un panel de control para gestionar vehículos, categorías, características y usuarios.

La aplicación ha sido construida utilizando arquitecturas modernas y se encuentra totalmente dockerizada, permitiendo un despliegue rápido y consistente en cualquier servidor que cuente con los requisitos mínimos. La versión de producción v1.0.0 fue lanzada el 19 de junio de 2026 y está disponible en autoreserve.duckdns.org.

Módulos Principales

- **Frontend:** Interfaz de usuario responsiva basada en React con navegación intuitiva
- **Backend:** API REST desarrollada en Spring Boot con servicios de negocio
- **Base de Datos:** MySQL para persistencia de datos
- **Infraestructura:** Docker y Docker Compose para orquestación de contenedores

2. Objetivos Del Sistema

Objetivo General

Desarrollar una aplicación web integral de gestión de reservas de vehículos que permita a diferentes tipos de usuarios (anónimos, registrados y administradores) interactuar con un catálogo de vehículos, realizar reservas y gestionar su perfil de manera segura y eficiente.

Objetivos Específicos

- Permitir que usuarios anónimos naveguen y visualicen el catálogo de vehículos disponibles
- Facilitar el registro e inicio de sesión para usuarios
- Implementar un sistema seguro de reservas con validación de disponibilidad
- Proporcionar a usuarios autenticados la capacidad de marcar favoritos y consultar historial
- Permitir a administradores gestionar el catálogo completo de vehículos y categorías
- Implementar un sistema de notificaciones y comunicación con proveedores
- Asegurar la escalabilidad y mantenibilidad del sistema mediante arquitectura modular
- Garantizar la seguridad de datos y acceso mediante autenticación y autorización basada en roles

3. Descripción Del Sistema De Información Desarrollado

3.1 Características Funcionales

AutoReserve implementa las siguientes características principales:

Funcionalidades de Usuario Anónimo:

- Navegación por catálogo de vehículos
- Búsqueda y filtrado de vehículos
- Visualización de detalle de vehículos con galerías de imágenes
- Consulta de disponibilidad
- Registro e inicio de sesión

Funcionalidades de Usuario Registrado:

- Realizar reservas con selección de fechas
- Consultar historial de reservas
- Marcar vehículos como favoritos
- Gestionar lista de favoritos
- Dejar reseñas y calificaciones
- Actualizar perfil de usuario
- Recibir notificaciones por correo electrónico

Funcionalidades de Administrador:

- Gestión completa de catálogo de vehículos (CRUD)
- Gestión de categorías de vehículos
- Gestión de características y atributos
- Administración de usuarios y asignación de roles
- Consulta de reservas y disponibilidad
- Generación de reportes

3.2 Requisitos Mínimos de Instalación

Para desplegar AutoReserve en un servidor, se requieren los siguientes componentes:

Hardware:

- CPU: 2 núcleos mínimo (recomendado 4)
- RAM: 4 GB mínimo (recomendado 8 GB)
- Almacenamiento: 20 GB disponibles (SSD recomendado)
- Ancho de banda: 100 Mbps mínimo

Software:

- Sistema Operativo: Ubuntu Server 20.04 LTS o superior
- Git: Versión 2.25 o superior
- Docker: Versión 20.10 o superior
- Docker Compose: Versión 1.29 o superior

Red:

- Acceso a internet para descargar dependencias
- Puerto 80 (HTTP) disponible para frontend
- Puerto 8080 disponible para backend
- Puerto 3306 disponible para base de datos (solo internamente en Docker)

Especificaciones recomendadas:

Recurso	Requerimiento
Sistema operativo	Ubuntu Server 24.04 LTS
Procesador	2 vCPU
Memoria RAM	8 GB
Almacenamiento	100 GB SSD
Docker	Instalado
Docker Compose	Instalado
Conectividad	Acceso a Internet

3.3 Actores del Sistema

Actor	Descripción	Permisos
Usuario Anónimo	Navega la aplicación sin autenticarse	Consulta de catálogo, búsqueda, visualización de detalles
Usuario Registrado	Usuario autenticado en el sistema	Todas las funciones de anónimo + reservas, favoritos, reseñas, historial
Administrador	Usuario con permisos administrativos	Gestión completa de datos, usuarios, categorías y configuración
Sistema Externo	Servicios de notificación y pago	Integración API para envío de correos y procesamiento de pagos

4. Diseño Técnico Del Sistema De Información

4.1 Stack Tecnológico y Frameworks

AutoReserve ha sido construido con las siguientes tecnologías:

Componente	Tecnología	Versión	Descripción
Frontend	React	19	Biblioteca para construir interfaces de usuario
Estilos	Tailwind CSS	Última	Framework CSS utility-first para diseño responsivo
Estado Frontend	Context API + Hooks	Nativa	Gestión de estado de aplicación sin Redux
Backend	Spring Boot	3.4.1	Framework Java para aplicaciones web empresariales
Java	JDK	21	Lenguaje de programación principal del backend
Base de Datos	MySQL	8.0	Sistema de gestión de base de datos relacional

Componente	Tecnología	Versión	Descripción
Autenticación	JWT + Spring Security	6.x	Seguridad y autenticación basada en tokens JWT
Validación	Spring Validation	3.4.1	Validación de datos de entrada
Containerización	Docker	20.10+	Plataforma de contenerización de aplicaciones
Orquestación	Docker Compose	1.29+	Herramienta para orquestar múltiples contenedores
Control de Versiones	Git	2.25+	Sistema de control de versiones distribuido
Servidor Web	Nginx	Última	Servidor web para servir frontend en Docker
Gestor de Dominio	DuckDNS	-	Asignación de dominio dinámico (autoreserve.duckdns.org)

4.2 Estándares de Codificación

El proyecto sigue los siguientes estándares internacionales:

Backend (Java/Spring Boot):

- Nomenclatura PascalCase para clases
- camelCase para variables, métodos y propiedades
- Separación en capas: Controller → Service → Repository → Entity
- Patrón DAO (Data Access Object) para acceso a datos
- API REST siguiendo principios RESTful
- Manejo de excepciones con Custom Exceptions
- Validación de datos con anotaciones de Spring

Frontend (React):

- PascalCase para componentes funcionales
- camelCase para funciones, variables y métodos
- Componentes funcionales con React Hooks

- Estructura: components → pages → services → context → utils
- Estilos con Tailwind CSS (utility-first)
- Props typing con PropTypes o TypeScript (cuando aplique)

Base de Datos:

- Nomenclatura snake_case para tablas y columnas
- Claves primarias: id (tipo BIGINT)
- Claves foráneas con sufijo _id
- Timestamps: created_at, updated_at (DATETIME)
- Índices en claves primarias, foráneas y campos de búsqueda frecuente

4.3 Modelo de Datos

4.3.1 Diagrama Entidad-Relación

El sistema gestiona las siguientes entidades principales con relaciones normalizadas:

Entidades Principales:

User	→	UserProfile	(relación 1:1)
User	→	Reservation	(relación 1:N)
User	→	Review	relación 1:N)
User	→	Favorite	(relación 1:N)
Car	→	Notification	(relación 1:N)
Car	→	Category	(relación N:1)
Car	→	Reservation	(relación 1:N)
Car	→	Review	(relación 1:N)
Car	→	Favorite	(relación 1:N)
Car	→	Branch	(relación N:1)
Reservation	→	Payment	(relación 1:1)

4.3.2 Diccionario de Datos Completo

Diccionario de Datos - AutoReserve

Tabla: branch

Descripción: Almacena la información de las sucursales donde los vehículos pueden ser recogidos y entregados por los clientes.

Campo	Tipo	PK/FK	Nulos	Descripción
id	BIGINT	PK	No	Identificador único de la sucursal.
address	VARCHAR(255)	-	Sí	Dirección física de la sucursal.
city	VARCHAR(100)	-	Sí	Ciudad donde se encuentra ubicada la sucursal.
image	VARCHAR(500)	-	Sí	URL o ruta de la imagen representativa de la sucursal.
name	VARCHAR(100)	-	No	Nombre comercial de la sucursal.
phone	VARCHAR(30)	-	Sí	Número telefónico de contacto de la sucursal.

Tabla: car

Descripción: Contiene la información de cada vehículo disponible para alquiler.

Campo	Tipo	PK/FK	Nulos	Descripción
id	BIGINT	PK	No	Identificador único del vehículo.
color	VARCHAR(50)	-	Sí	Color del vehículo.
notes	VARCHAR(500)	-	Sí	Observaciones internas del vehículo (estado, daños menores, etc.).
plate	VARCHAR(20)	Única	Sí	Placa del vehículo. Debe ser única en el sistema.
status	ENUM	-	No	Estado operativo del vehículo (AVAILABLE, RENTED, MAINTENANCE, etc.).

Campo	Tipo	PK/FK	Nulos	Descripción
branch_id	BIGINT	FK	No	Sucursal donde se encuentra actualmente el vehículo.
car_model_id	BIGINT	FK	No	Modelo al que pertenece el vehículo.

Tabla: car_model

Descripción

Almacena la información general de cada modelo de vehículo que ofrece la empresa.

Campo	Tipo	PK/FK	Nulos	Descripción
id	BIGINT	PK	No	Identificador del modelo de vehículo.
brand	VARCHAR(100)	-	No	Marca del vehículo (Toyota, Mazda, Renault, etc.).
description	VARCHAR(1000)	-	Sí	Descripción general del modelo.
image	VARCHAR(500)	-	Sí	Imagen representativa del modelo.
model	VARCHAR(100)	-	No	Nombre o referencia del modelo.
price_per_day	DECIMAL(10,2)	-	No	Precio base por día de alquiler.
year	INT	-	Sí	Año del modelo del vehículo.
category_id	BIGINT	FK	No	Categoría a la que pertenece el modelo.

Tabla: category

Descripción: Define las categorías de vehículos disponibles para alquiler.

Campo	Tipo	PK/FK	Nulos	Descripción
id	BIGINT	PK	No	Identificador único de la categoría.
description	TEXT	-	Sí	Descripción de la categoría.
image	VARCHAR(255)	-	Sí	Imagen representativa de la categoría.
name	VARCHAR(100)	-	No	Nombre de la categoría (SUV, Sedán, Pickup, etc.).

Tabla: contact_message

Descripción: Registra los mensajes enviados por los usuarios mediante el formulario de contacto.

Campo	Tipo	PK/FK	Nulos	Descripción
id	BIGINT	PK	No	Identificador del mensaje.
created_at	DATETIME	-	No	Fecha de creación del mensaje.
sender_email	VARCHAR(200)	-	No	Correo electrónico del remitente.
sender_name	VARCHAR(100)	-	No	Nombre del remitente.
status	ENUM	-	No	Estado del mensaje (OPEN, IN_PROGRESS, ANSWERED, CLOSED).
subject	VARCHAR(200)	-	No	Asunto del mensaje.
type	ENUM	-	No	Tipo de solicitud (Pregunta, Queja, Reclamo, Solicitud o Felicitación).
updated_at	DATETIME	-	Sí	Fecha de la última actualización del mensaje.
user_id	BIGINT	FK	Sí	Usuario que envió el mensaje, si se encontraba autenticado.

Tabla: feature

Descripción: Almacena las características o funcionalidades que pueden estar asociadas a uno o varios modelos de vehículos, permitiendo describir sus prestaciones y equipamiento.

Campo	Tipo	PK/FK	Nulos	Descripción
id	BIGINT	PK	No	Identificador único de la característica.
description	VARCHAR(500)	-	Sí	Descripción detallada de la característica del vehículo.
icon	VARCHAR(255)	-	Sí	Nombre o ruta del ícono utilizado para representar visualmente la característica.

Campo	Tipo	PK/FK	Nulos	Descripción
name	VARCHAR(100)	-	No	Nombre de la característica (por ejemplo: Aire acondicionado, GPS, Bluetooth, Airbags, etc.).

Tabla: feature_car_model

Descripción: Tabla intermedia que implementa la relación muchos a muchos entre los modelos de vehículos y sus características.

Campo	Tipo	PK/FK	Nulos	Descripción
feature_id	BIGINT	PK / FK	No	Identificador de la característica asociada al modelo del vehículo.
car_model_id	BIGINT	PK / FK	No	Identificador del modelo de vehículo al que pertenece la característica.

Tabla: payment

Descripción: Registra la información correspondiente a los pagos realizados por las reservas efectuadas dentro del sistema.

Campo	Tipo	PK/FK	Nulos	Descripción
id	BIGINT	PK	No	Identificador único del pago.
amount	DECIMAL(10,2)	-	No	Valor total pagado por la reserva.
payment_date	DATETIME	-	No	Fecha y hora en que se realizó el pago.
payment_method	ENUM	-	No	Medio de pago utilizado (Tarjeta de crédito, débito, transferencia, entre otros).
status	ENUM	-	No	Estado actual del pago (Pendiente, Aprobado, Rechazado o Reembolsado).

Campo	Tipo	PK/FK	Nulos	Descripción
transaction_reference	VARCHAR(255)	-	Sí	Código o referencia generada por la pasarela de pagos para identificar la transacción.
reservation_id	BIGINT	FK	No	Reserva a la cual corresponde el pago registrado.

Tabla: reservation

Descripción: Almacena la información de las reservas realizadas por los clientes para el alquiler de vehículos.

Campo	Tipo	PK/FK	Nulos	Descripción
id	BIGINT	PK	No	Identificador único de la reserva.
created_at	DATETIME	-	No	Fecha y hora en que se registró la reserva.
end_date	DATE	-	No	Fecha de finalización del período de alquiler.
pickup_time	TIME	-	No	Hora programada para recoger el vehículo.
return_time	TIME	-	No	Hora programada para devolver el vehículo.
start_date	DATE	-	No	Fecha de inicio del alquiler.
status	ENUM	-	No	Estado actual de la reserva (Pendiente, Confirmada, Cancelada, Finalizada, etc.).
total_price	DECIMAL(10,2)	-	No	Valor total calculado para la reserva según el período de alquiler.
branch_pickup_id	BIGINT	FK	No	Sucursal donde el cliente recogerá el vehículo.

Campo	Tipo	PK/FK	Nulos	Descripción
branch_return_id	BIGINT	FK	No	Sucursal donde el cliente devolverá el vehículo.
car_id	BIGINT	FK	No	Vehículo reservado por el cliente.
user_id	BIGINT	FK	No	Usuario que realizó la reserva.

Tabla: review

Descripción: Almacena las reseñas y calificaciones realizadas por los usuarios sobre los vehículos que han reservado, permitiendo conocer el nivel de satisfacción con el servicio.

Campo	Tipo	PK/FK	Nulos	Descripción
id	BIGINT	PK	No	Identificador único de la reseña.
comment	TEXT	-	Sí	Comentario realizado por el usuario sobre el vehículo o la experiencia del servicio.
created_at	DATETIME(6)	-	No	Fecha y hora en que fue registrada la reseña.
rating	INT	-	No	Calificación otorgada por el usuario al vehículo o al servicio.
car_id	BIGINT	FK	No	Vehículo al que corresponde la reseña.
user_id	BIGINT	FK	No	Usuario que realizó la reseña.

Tabla: role

Descripción: Almacena los diferentes roles disponibles en el sistema para definir los permisos y niveles de acceso de los usuarios.

Campo	Tipo	PK/FK	Nulos	Descripción
id	BIGINT	PK	No	Identificador único del rol.
name	VARCHAR(50)	-	No	Nombre del rol asignado al usuario (por ejemplo, Administrador o Cliente).

Tabla: user

Descripción: Contiene la información básica de autenticación e identificación de los usuarios registrados en la plataforma.

Campo	Tipo	PK/FK	Nulos	Descripción
id	BIGINT	PK	No	Identificador único del usuario.
created_at	DATETIME(6)	-	No	Fecha y hora de creación de la cuenta.
email	VARCHAR(150)	-	No	Dirección de correo electrónico utilizada para iniciar sesión.
first_name	VARCHAR(100)	-	No	Nombre del usuario.
last_name	VARCHAR(100)	-	Sí	Apellido del usuario.
password_hash	VARCHAR(255)	-	No	Contraseña almacenada de forma cifrada mediante hash.
phone	VARCHAR(30)	-	Sí	Número telefónico del usuario.
status	ENUM('ACTIVE','INACTIVE','SUSPENDED')	-	Sí	Estado actual de la cuenta del usuario.
role_id	BIGINT	FK	No	Rol asignado al usuario dentro del sistema.

Tabla: user_profile

Descripción: Almacena información adicional del perfil de cada usuario que complementa los datos básicos de autenticación.

Campo	Tipo	PK/FK	Nulos	Descripción
id	BIGINT	PK	No	Identificador único del perfil del usuario.

Campo	Tipo	PK/FK	Nulos	Descripción
address	VARCHAR(255)	-	Sí	Dirección de residencia del usuario.
birth_date	DATE	-	Sí	Fecha de nacimiento del usuario.
city	VARCHAR(100)	-	Sí	Ciudad de residencia del usuario.
driving_license	VARCHAR(100)	-	Sí	Número o identificación de la licencia de conducción del usuario.
user_id	BIGINT	FK	No	Usuario propietario del perfil. Cada usuario posee un único perfil.

Tabla: faq

Descripción: Almacena las preguntas frecuentes publicadas en la plataforma para orientar a los usuarios sobre el funcionamiento del servicio.

Campo	Tipo	PK/FK	Nulos	Descripción
id	BIGINT	PK	No	Identificador único de la pregunta frecuente.
answer	TEXT	-	No	Respuesta correspondiente a la pregunta frecuente.
question	VARCHAR(300)	-	No	Pregunta que será mostrada al usuario.
sort_order	INT	-	No	Orden en el que se presenta la pregunta dentro de la lista.
updated_at	DATETIME(6)	-	No	Fecha y hora de la última actualización del registro.

Tabla: favorite

Descripción: Registra los modelos de vehículos marcados como favoritos por los usuarios para facilitar su consulta posterior.

Campo	Tipo	PK/FK	Nulos	Descripción
id	BIGINT	PK	No	Identificador único del registro de favorito.

Campo	Tipo	PK/FK	Nulos	Descripción
created_at	DATETIME(6)	-	No	Fecha y hora en que el vehículo fue agregado a favoritos.
car_model_id	BIGINT	FK	No	Modelo de vehículo marcado como favorito.
user_id	BIGINT	FK	No	Usuario propietario del registro de favorito.

Tabla: invoice

Descripción: Almacena la información de las facturas generadas a partir de los pagos realizados por los usuarios, sirviendo como soporte del proceso de facturación del sistema.

Campo	Tipo	PK/FK	Nulos	Descripción
id	BIGINT	PK	No	Identificador único de la factura.
details	TEXT	-	Sí	Información adicional o detalle de los conceptos incluidos en la factura.
invoice_number	VARCHAR(50)	-	No	Número único que identifica la factura emitida.
issue_date	DATETIME(6)	-	Sí	Fecha y hora de emisión de la factura.
total_amount	DECIMAL(10,2)	-	Sí	Valor total facturado correspondiente al pago realizado.
payment_id	BIGINT	FK	No	Identificador del pago asociado a la factura. Cada pago genera una única factura.

Tabla: message_reply

Descripción: Almacena las respuestas realizadas a los mensajes enviados mediante el formulario de contacto, permitiendo mantener un historial de la comunicación entre el cliente y la administración.

Campo	Tipo	PK/FK	Nulos	Descripción
id	BIGINT	PK	No	Identificador único de la respuesta.
author_name	VARCHAR(100)	-	No	Nombre de la persona que envía la respuesta.
content	TEXT	-	No	Contenido del mensaje de respuesta.
created_at	DATETIME(6)	-	No	Fecha y hora en que se registró la respuesta.
sent_by	ENUM('ADMIN','CLIENT')	-	No	Indica si la respuesta fue enviada por un administrador o por un cliente.
contact_message_id	BIGINT	FK	No	Mensaje de contacto al que pertenece la respuesta.

Tabla: notification

Descripción: Almacena las notificaciones enviadas a los usuarios para informar eventos importantes relacionados con reservas, pagos y otras actividades del sistema.

Campo	Tipo	PK/FK	Nulos	Descripción
id	BIGINT	PK	No	Identificador único de la notificación.
message	TEXT	-	No	Contenido del mensaje que recibirá el usuario.
read_flag	BIT(1)	-	No	Indica si la notificación ha sido leída por el usuario.
sent_at	DATETIME(6)	-	Sí	Fecha y hora en que la notificación fue enviada.
type	ENUM('ALERT','PAYMENT','RESERVATION')	-	No	Tipo de notificación generada por el sistema.

Campo	Tipo	PK/FK	Nulos	Descripción
user_id	BIGINT	FK	No	Usuario destinatario de la notificación.

Tabla: policy

Descripción: Almacena las políticas publicadas por la empresa, como políticas de privacidad, términos y condiciones, políticas de cancelación y demás información institucional.

Campo	Tipo	PK/FK	Nulos	Descripción
id	BIGINT	PK	No	Identificador único de la política.
content	TEXT	-	No	Contenido completo de la política.
slug	VARCHAR(50)	-	No	Identificador único utilizado para acceder a la política mediante una URL amigable.
sort_order	INT	-	No	Orden en el que se muestra la política dentro de la aplicación.
title	VARCHAR(200)	-	No	Título de la política publicada.
updated_at	DATETIME(6)	-	No	Fecha y hora de la última actualización de la política.

4.4 Arquitectura de Componentes

La arquitectura de AutoReserve se organiza en tres capas principales:

4.4.1 Capa Frontend (React 19 + Tailwind CSS)

Componentes Principales:

- **Navigation:** Barra de navegación superior con links de login/registro y menú de usuario
- **Home:** Página de inicio con catálogo aleatorio de vehículos
- **CarDetail:** Página de detalle de vehículo con galerías e información completa

- **Catalog:** Página con listado completo de vehículos con búsqueda y filtros
- **Forms:** Componentes de formularios para registro, login y reserva
- **Modals:** Componentes modales para confirmaciones y detalles
- **Cards:** Componentes visuales para mostrar vehículos en grid
- **Admin Panel:** Sección administrativa para gestión de datos

Servicios Frontend:

- **API Client:** Cliente HTTP (fetch/axios) para comunicación con backend
- **Auth Service:** Gestión de autenticación y tokens JWT
- **Car Service:** Operaciones CRUD de vehículos
- **Reservation Service:** Gestión de reservas
- **User Service:** Operaciones de perfil de usuario

Gestión de Estado:

- **Auth Context:** Estado global de autenticación y datos de usuario
- **Car Context:** Estado del catálogo de vehículos
- **React Router:** Navegación entre páginas

4.4.2 Capa Backend (Spring Boot 3.4.1 + Java 21)

REST Controllers:

- **AuthController:** Endpoints de registro (POST /api/auth/register) y login (POST /api/auth/login)
- **CarController:** CRUD completo de vehículos (GET, POST, PUT, DELETE /api/cars)
- **ReservationController:** Gestión de reservas (crear, listar, actualizar, cancelar)
- **UserController:** Gestión de perfiles de usuario
- **ReviewController:** Operaciones de reseñas y calificaciones
- **CategoryController:** Gestión de categorías de vehículos
- **FavoriteController:** Gestión de favoritos
- **AdminController:** Endpoints administrativos especiales

Servicios de Negocio:

- **UserService:** Lógica de creación, autenticación y gestión de usuarios
- **CarService:** Lógica de vehículos, disponibilidad y búsqueda
- **ReservationService:** Lógica de reservas, cálculo de precios y validaciones
- **PaymentService:** Procesamiento y validación de pagos
- **NotificationService:** Envío de notificaciones por correo electrónico
- **ReviewService:** Gestión de reseñas y cálculo de promedios

Repositorios (Data Access Object):

- **UserRepository:** Acceso a datos de usuarios
- **CarRepository:** Acceso a datos de vehículos con búsquedas personalizadas
- **ReservationRepository:** Acceso a reservas con filtros de fecha
- **ReviewRepository:** Acceso a reseñas
- **FavoriteRepository:** Acceso a favoritos

Seguridad:

- **JwtService:** Generación, validación y renovación de tokens JWT
- **JwtAuthenticationFilter:** Interceptor de autenticación en cada request
- **SecurityConfig:** Configuración de permisos por rol y endpoints
- **AuthFilter:** Filtro para validar tokens en requests

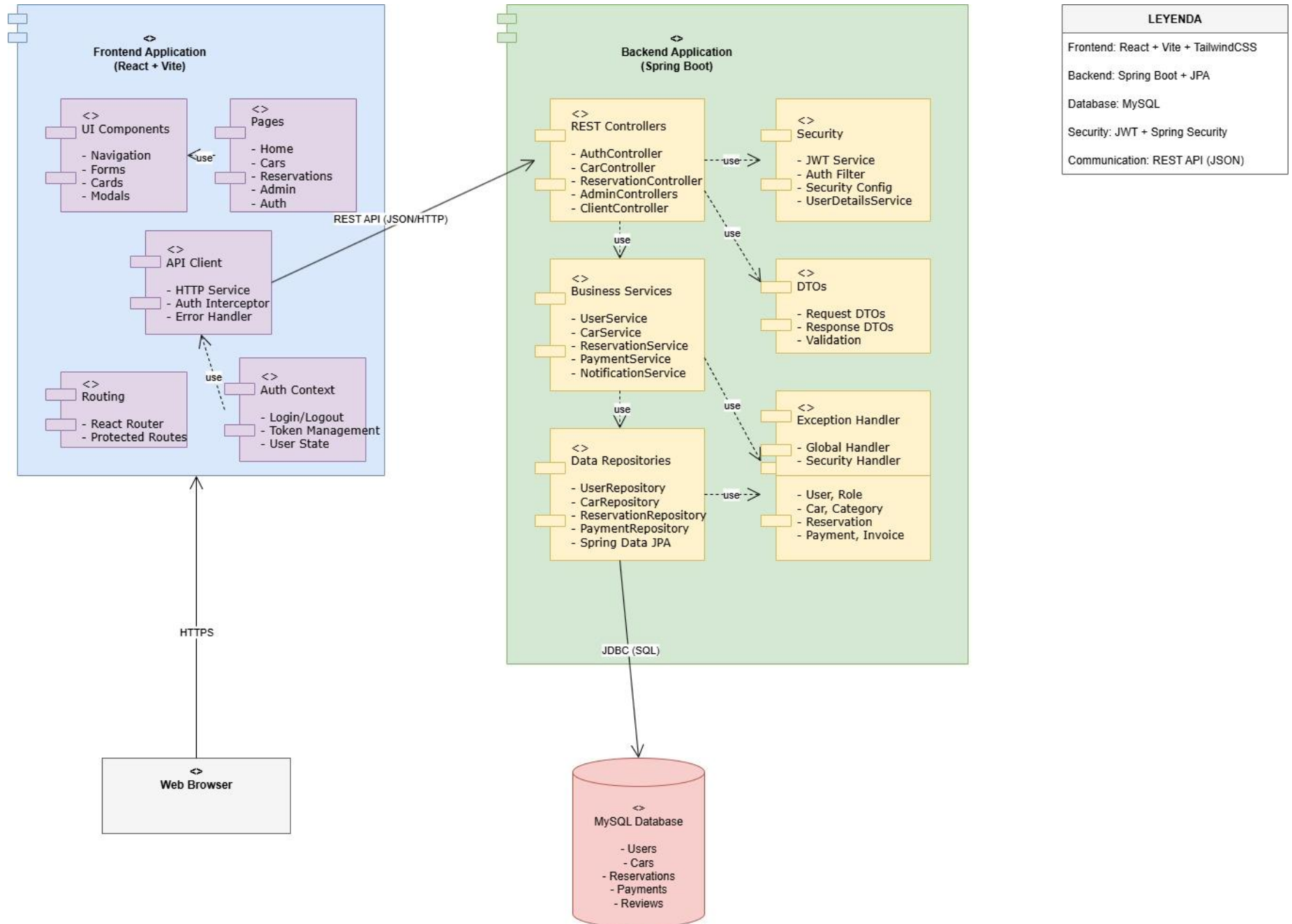
DTOs (Data Transfer Objects):

- **Request DTOs:** UserRegisterRequest, LoginRequest, CarRequest, ReservationRequest
- **Response DTOs:** UserResponse, CarResponse, ReservationResponse, ErrorResponse

4.4.3 Capa Base de Datos (MySQL 8.0)

- 11 tablas principales normalizadas
- Relaciones establecidas con restricciones de integridad referencial
- Índices creados en claves primarias, foráneas y campos de búsqueda frecuente
- Triggers para auditoría de cambios (opcional)
- Vistas para reportes (opcional)

Diagrama de componentes.



Esquema Diagrama de Clases.

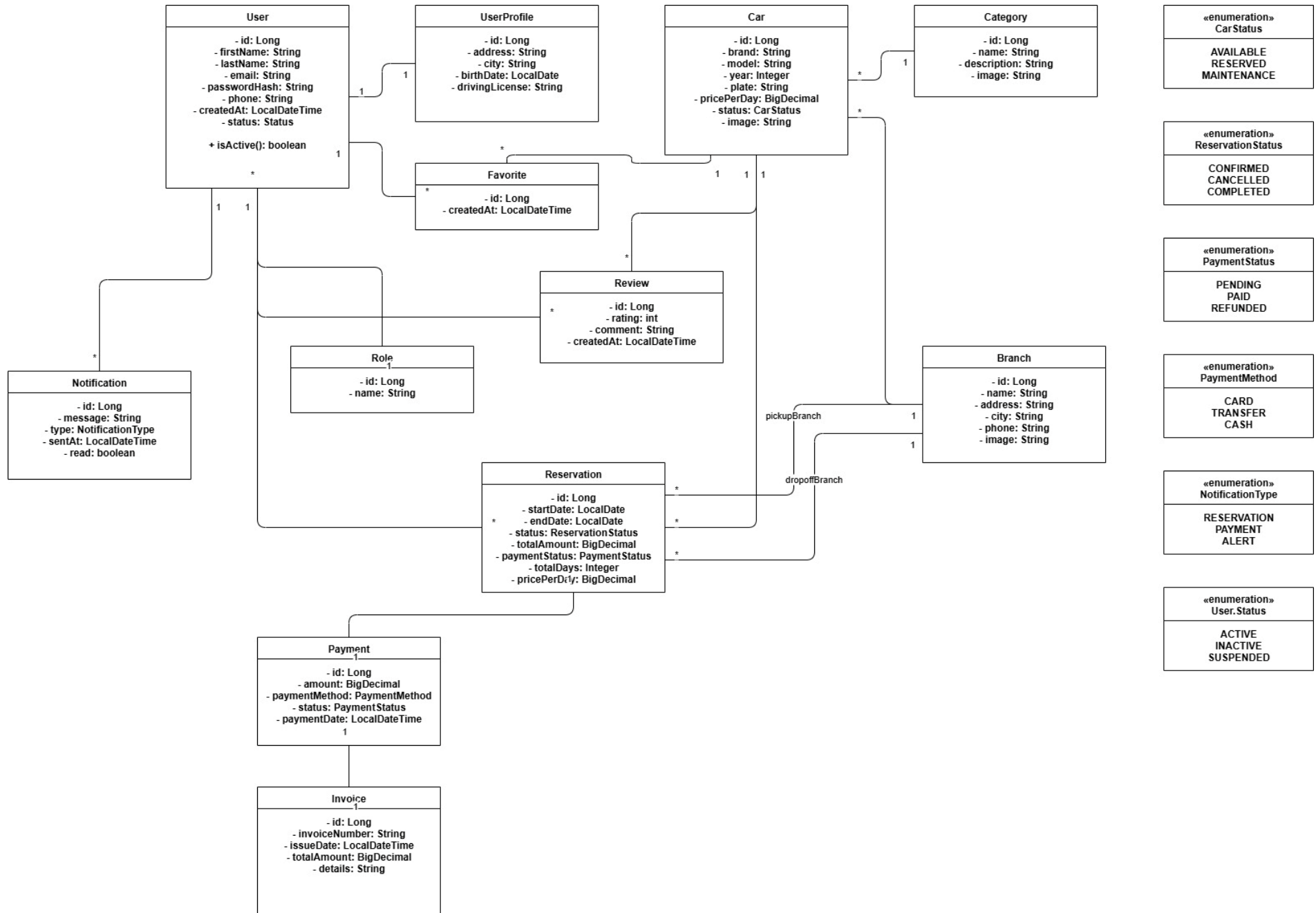
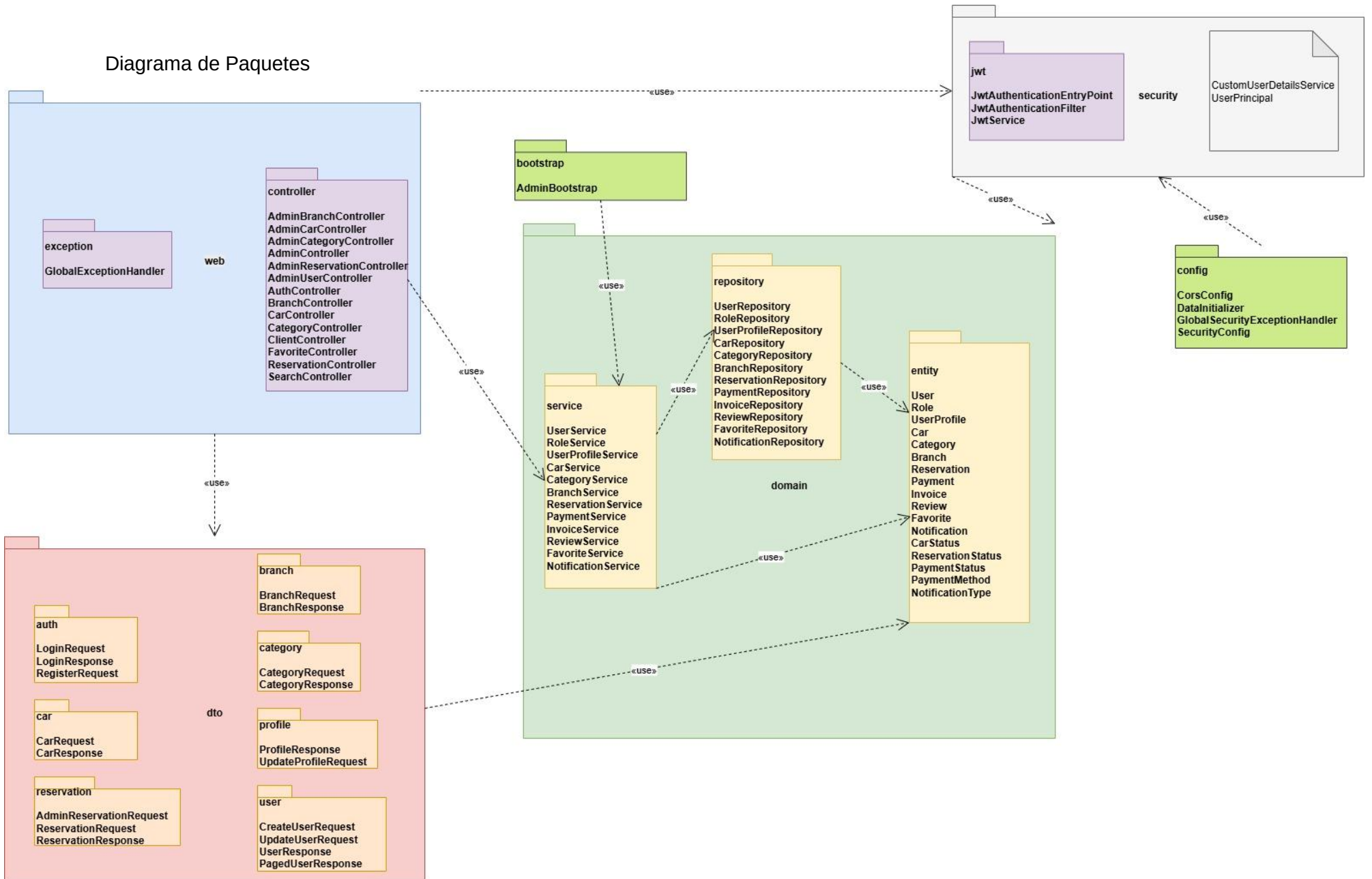
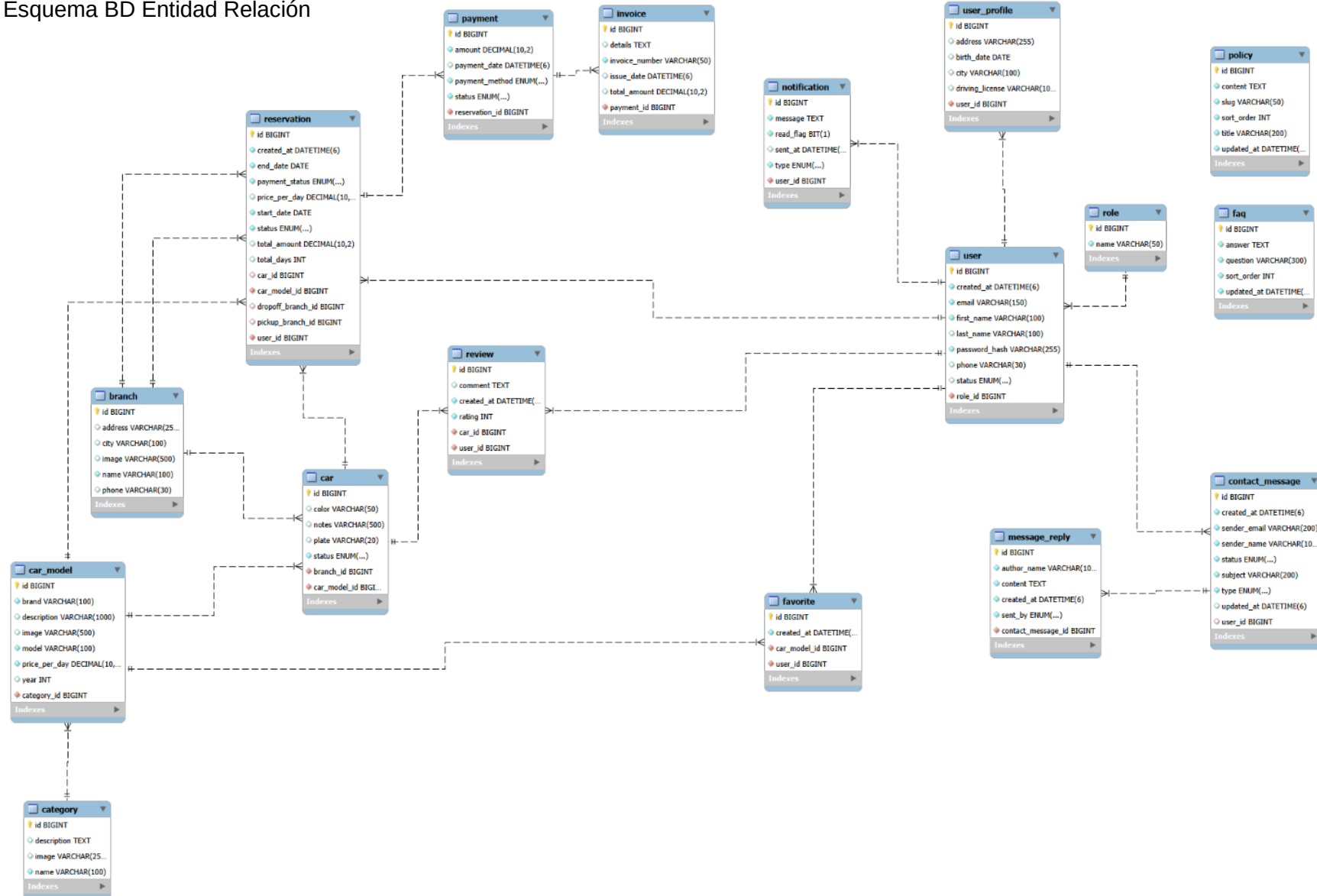


Diagrama de Paquetes



Esquema BD Entidad Relación



4.5 Casos de Uso del Sistema

ID	Caso de Uso	Actor	Descripción
UC1	Navegar por el catálogo de autos	Usuario Anónimo/Registrado	Permite explorar la lista de autos disponibles con paginación y búsqueda básica
UC2	Visualizar detalle de auto	Usuario Anónimo/Registrado	Acceder a información detallada de un vehículo (imágenes, descripción, características, reseñas)
UC3	Registrarse en la aplicación	Usuario Anónimo	Completar un formulario para crear una cuenta con validaciones de email único
UC4	Iniciar sesión	Usuario Registrado	Autenticarse con email y contraseña para acceder a funcionalidades adicionales
UC5	Realizar reserva de auto	Usuario Registrado	Seleccionar un vehículo, elegir fechas disponibles y confirmar reserva con cálculo automático
UC6	Consultar historial de reservas	Usuario Registrado	Ver todas las reservas realizadas anteriormente con detalles y estados
UC7	Marcar auto como favorito	Usuario Registrado	Guardar autos en una lista de favoritos para acceso rápido desde perfil
UC8	Gestionar autos	Administrador	Crear, editar y eliminar autos en el catálogo incluyendo carga de imágenes
UC9	Gestionar categorías y características	Administrador	Definir categorías de autos (SUV, Sedán, etc.) y características específicas
UC10	Gestionar usuarios	Administrador	Asignar roles de administrador y controlar usuarios activos/suspendidos

4.6 Servicios y APIs Expuestos

Endpoints REST Principales

Autenticación:

POST /api/auth/register - Registro de nuevo usuario
POST /api/auth/login - Inicio de sesión
POST /api/auth/refresh - Renovación de token JWT
POST /api/auth/logout - Cierre de sesión

Vehículos:

GET /api/cars - Listar todos los vehículos con paginación
GET /api/cars/{id} - Obtener detalle de vehículo específico
POST /api/cars - Crear nuevo vehículo (Admin)
PUT /api/cars/{id} - Actualizar vehículo (Admin)
DELETE /api/cars/{id} - Eliminar vehículo (Admin)
GET /api/cars/search - Buscar vehículos por criterios (marca, modelo, rango precio)
GET /api/cars/random - Obtener vehículos aleatorios para home

Reservas:

POST /api/reservations - Crear nueva reserva
GET /api/reservations/{id} - Obtener detalle de reserva específica
GET /api/reservations/user/{userId} - Historial de reservas del usuario
PUT /api/reservations/{id} - Actualizar estado de reserva
DELETE /api/reservations/{id} - Cancelar reserva
GET /api/reservations/available-dates/{carId} - Consultar disponibilidad

Usuarios:

GET /api/users/{id} - Obtener perfil de usuario
PUT /api/users/{id} - Actualizar perfil (nombre, teléfono, dirección)
GET /api/users - Listar usuarios (Admin)
DELETE /api/users/{id} - Eliminar usuario (Admin)

Reseñas:

POST /api/reviews - Crear nueva reseña
GET /api/reviews/car/{carId} - Obtener reseñas de vehículo específico
PUT /api/reviews/{id} - Actualizar reseña propia
DELETE /api/reviews/{id} - Eliminar reseña propia
GET /api/reviews/stats/{carId} - Obtener estadísticas de reseñas

Favoritos:

POST /api/favorites - Agregar vehículo a favoritos
GET /api/favorites - Obtener lista de favoritos del usuario
DELETE /api/favorites/{id} - Remover vehículo de favoritos

Categorías:

GET /api/categories - Listar todas las categorías
POST /api/categories - Crear nueva categoría (Admin)
PUT /api/categories/{id} - Actualizar categoría (Admin)
DELETE /api/categories/{id} - Eliminar categoría (Admin)

5. Despliegue Y Configuración De Componentes

5.1 Archivos de Configuración

5.1.1 Archivo .env

El archivo .env contiene variables de entorno necesarias para la ejecución. Se debe crear en la raíz del proyecto con el siguiente contenido:

```
# =====  
# MYSQL  
# =====  
MYSQL_ROOT_PASSWORD=1234root  
MYSQL_DATABASE=autoreserve_app_bd  
MYSQL_USER=autoreserve  
MYSQL_PASSWORD=1234autoreserve
```

```
# =====
# SPRING BOOT
# =====

SPRING_PROFILES_ACTIVE=default


# =====
# PUERTOS
# =====

BACKEND_PORT=8080
FRONTEND_PORT=80
MYSQL_PORT=3306
```

Descripción de variables:

Variable	Descripción	Valor por Defecto
MYSQL_ROOT_PASSWORD	Contraseña del usuario root de MySQL	1234root
MYSQL_DATABASE	Nombre de la base de datos a crear	autoreserve_app_bd
MYSQL_USER	Usuario de aplicación para MySQL	autoreserve
MYSQL_PASSWORD	Contraseña del usuario de aplicación	1234autoreserve
SPRING_PROFILES_ACTIVE	Perfil activo de Spring	default
BACKEND_PORT	Puerto en el que corre el backend	8080
FRONTEND_PORT	Puerto en el que corre el frontend	80
MYSQL_PORT	Puerto de MySQL (interno)	3306

5.1.2 Archivo application.properties

Ubicado en backend/src/main/resources/application.properties, contiene la configuración de Spring Boot:

```
spring.application.name=autoreserve-backend
```

```
#
=====
# CONFIGURACIÓN DE PRODUCCIÓN/DOCKER
#
=====
# Esta configuración está pensada para Docker Compose donde 'mysql' es el service name
# Para desarrollo local, usa: --spring.profiles.active=dev
# Para tests, usa: --spring.profiles.active=test

# Base de datos MySQL para producción/Docker
spring.datasource.url=jdbc:mysql://mysql:3306/autoreserve_app_bd?useSSL=false&serverTimezone=UTC
spring.datasource.username=autoreserve
spring.datasource.password=1234autoreserve
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# Configuración JPA para producción
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false
spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
spring.jpa.properties.hibernate.format_sql=true

# Configuración de vistas
spring.mvc.view.prefix=/WEB-INF/views/
spring.mvc.view.suffix=.jsp

# Logging para producción (menos verbose)
logging.level.com.autoreserve.backend=INFO
logging.level.org.springframework.web=WARN
logging.level.org.hibernate.SQL=WARN

# JMX deshabilitado en producción por seguridad
spring.jmx.enabled=false
```

Configuración de sesión

server.servlet.session.timeout=30m

server.servlet.session.cookie.http-only=true

server.servlet.session.cookie.secure=false

Límites de upload

spring.servlet.multipart.max-file-size=10MB

spring.servlet.multipart.max-request-size=10MB

Nota importante: En producción, cambiar `server.servlet.session.cookie.secure=true` cuando se implemente HTTPS.

5.2 Requisitos Previos de Instalación

Antes de desplegar AutoReserve, asegúrate de contar con:

En el servidor (Instancia EC2):

- Sistema operativo: Ubuntu Server 24.04 LTS
- Acceso SSH configurado correctamente
- Usuario con permisos sudo habilitados
- Conexión a internet estable (para descargar dependencias)
- Puertos 80 (HTTP), 8080 (backend) y 3306 (MySQL interno) disponibles

En tu máquina local:

- Cliente SSH (PuTTY, OpenSSH, Terminal, etc.)
- Navegador web moderno para acceder a la aplicación

Recurso	Requerimiento
Sistema operativo	Ubuntu Server 24.04 LTS
Procesador	2 vCPU
Memoria RAM	8 GB
Almacenamiento	100 GB SSD
Docker	Instalado

Recurso	Requerimiento
Docker Compose	Instalado
Conectividad	Acceso a Internet

5.3 Instalación de Dependencias en el Servidor

Paso 1: Actualizar el Sistema

```
sudo apt update && sudo apt upgrade -y
```

Este comando actualiza los índices de paquetes e instala todas las actualizaciones de seguridad y parches disponibles.

Paso 2: Instalar Git

```
sudo apt install -y git
git --version
```

Git es necesario para clonar el repositorio de AutoReserve desde GitHub. El comando `git -version` verifica que la instalación fue exitosa.

Paso 3: Instalar Docker

Se utiliza el script oficial de instalación de Docker, que instala Docker Engine y el plugin de Docker Compose en un solo paso:

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
```

Este script automatiza toda la instalación incluyendo dependencias necesarias.

Paso 4: Permitir Ejecutar Docker sin sudo

Para simplificar los comandos posteriores, agrega tu usuario al grupo docker:

```
sudo usermod -aG docker $USER
newgrp docker
```

Nota: Después de ejecutar este comando, es recomendable cerrar la sesión SSH y volver a conectarse para que los cambios tomen efecto.

Paso 5: Verificar las Instalaciones

```
docker --version
docker compose version
git --version
```

Todos estos comandos deben retornar las versiones instaladas sin errores. Ejemplo de salida esperada:

```
Docker version 20.10.x, build xxxxxxxxx
Docker Compose version v2.x.x
git version 2.x.x
```

5.4 Clonado del Repositorio y Configuración

Paso 6: Clonar el Repositorio

```
git clone https://github.com/Leo-Quiroga/app-rentcar-sena.git
cd app-rentcar-sena
```

Este comando descarga el repositorio completo del proyecto en tu servidor. El cambio de directorio es importante para los pasos siguientes.

Nota importante: Si el repositorio es privado, GitHub solicitará autenticación usando usuario y un Personal Access Token (las contraseñas tradicionales ya no son aceptadas).

Paso 7: Crear el Archivo .env

El proyecto incluye un archivo `_env.example` con la estructura de variables necesarias. Crea un archivo `.env` basado en esa plantilla:

```
nano .env
```

Completa el archivo con el siguiente contenido (ajusta contraseñas según tus requisitos de seguridad):

```
# =====
# MYSQL
# =====
MYSQL_ROOT_PASSWORD=1234root
MYSQL_DATABASE=autoreserve_app_bd
MYSQL_USER=autoreserve
MYSQL_PASSWORD=1234autoreserve

# =====
# SPRING BOOT
# =====
SPRING_PROFILES_ACTIVE=default

# =====
# PUERTOS
# =====
BACKEND_PORT=8080
FRONTEND_PORT=80
MYSQL_PORT=3306
```

Guardar el archivo: Presiona Ctrl + O, luego Enter, y sal del editor con Ctrl + X.

5.5 Construcción y Despliegue de los Contenedores

Paso 8: Levantar los Servicios con Docker Compose

```
docker compose up -d --build
```

Parámetros explicados:

- -d: Ejecuta los contenedores en segundo plano (modo detached)
- --build: Fuerza la construcción de las imágenes del backend y frontend a partir de sus respectivos Dockerfile

Este comando puede tomar algunos minutos en su primera ejecución mientras descarga imágenes base y construye los contenedores.

Paso 9: Verificar el Estado de los Contenedores

`docker compose ps`

Deberías ver un output similar a:

NAME	COMMAND	SERVICE	STATUS	PORTS
autoreserve-mysql	"docker-entrypoint.sh"	mysql	Up 2 minutes	(healthy)
autoreserve-backend	"java -jar app.jar"	backend	Up 1 minute	0.0.0.0:8080->8080/tcp
autoreserve-frontend	"nginx -g daemon off"	frontend	Up 1 minute	0.0.0.0:80->80/tcp

Los tres servicios deben aparecer con estado "Up" o "running", y MySQL debe mostrarse como "healthy".

Paso 10: Revisar los Registros (Logs)

En caso de errores, puedes revisar los logs de cada servicio:

`docker compose logs -f mysql`

`docker compose logs -f backend`

`docker compose logs -f frontend`

Nota: Usa Ctrl + C para salir de la vista de logs en tiempo real.

Logs comunes esperados:

MySQL:

[Server] /usr/sbin/mysqld: ready for connections

Backend:

Started AutoReserveBackendApplication in x.xxx seconds (JVM running for x.xxx)

Frontend:

[notice] worker process started

5.6 Verificación del Acceso a la Aplicación

Paso 11: Obtener la IP Pública (Opcional)

Si necesitas conocer la IP pública con la que se accede desde internet:

```
curl ifconfig.me
```

Este comando devuelve la IP pública de tu instancia EC2 (en este caso, gestionada por AWS mediante NAT).

Paso 12: Acceder desde el Navegador

1. Abre un navegador web en tu máquina local
2. Ingresa la dirección: `http://autoreserve.duckdns.org/` o `http://<IP_PUBLICA>/`
3. Confirma que la interfaz del frontend de AutoReserve cargue correctamente
4. Opcionalmente, prueba el backend directamente en:
`http://autoreserve.duckdns.org:8080/` o `http://<IP_PUBLICA>:8080/`

Pruebas recomendadas:

- Navegar por el catálogo de vehículos
- Registrar un usuario nuevo
- Iniciar sesión
- Intentar crear una reserva
- Acceder al panel de administración (si tienes cuenta admin)

6. Resolución De Problemas

6.1 Errores Técnicos Más Comunes y Su Solución

Error 1: "Connection Refused" - El Backend no Responde

Síntomas:

- Al acceder a `http://localhost:8080` o `http://<IP>:8080` sale error de conexión

- Frontend intenta conectar pero no obtiene respuesta del backend

Causas Posibles:

- El contenedor del backend no está corriendo
- El puerto 8080 está ocupado por otra aplicación
- Spring Boot no ha iniciado correctamente debido a errores en la configuración
- Problema de conectividad entre contenedores Docker

Diagnóstico:

`docker compose ps | grep backend`

`docker compose logs backend`

Solución Paso a Paso:

1. Verificar si el contenedor está corriendo

`docker compose ps`

2. Si no está, levantarlo de nuevo

`docker compose up -d backend`

3. Esperar 30 segundos a que Spring Boot inicie

`sleep 30`

4. Revisar logs en detalle

`docker compose logs -f backend`

5. Si hay errores de base de datos, verificar MySQL está sano

`docker compose logs mysql`

6. Último recurso: reconstruir todos los contenedores

`docker compose down`

`docker compose up -d --build`

Verificación de solución:

curl http://localhost:8080/

Error 2: "Access Denied for User" - Problemas de Conectividad a MySQL

Síntomas:

- Backend muestra error: Access denied for user 'autoreserve'@'localhost'
- Logs de backend muestran excepciones de conectividad MySQL
- Las tablas no se crean automáticamente

Causas Posibles:

- Las credenciales en .env no coinciden con las de application.properties
- MySQL aún no ha terminado de inicializar
- La base de datos especificada en la URL JDBC no existe
- Problemas con el nombre del host (mysql vs localhost)

Diagnóstico:

Ver estado de MySQL

```
docker compose ps mysql
```

Ver logs de MySQL

```
docker compose logs mysql
```

Intentar conectar directamente a MySQL desde el contenedor backend

```
docker compose exec backend mysql -h mysql -u autoreserve -p1234autoreserve -e "SHOW DATABASES;"
```

Solución Paso a Paso:

1. Detener servicios

```
docker compose down
```

2. Verificar contenido de .env

```
cat .env
```

3. Verificar application.properties

cat backend/src/main/resources/application.properties | grep datasource

4. IMPORTANTE: En Docker Compose, el nombre del host es el nombre del servicio

En application.properties debe ser: jdbc:mysql://mysql:3306/...

NO: jdbc:mysql://localhost:3306/...

5. Levantar nuevamente

docker compose up -d --build

6. Esperar 45 segundos a que MySQL y Spring Boot inicien completamente

sleep 45

7. Verificar conexión

docker compose exec backend mysql -h mysql -u autoreserve -p1234autoreserve -e "SHOW TABLES;"

Verificación de solución:

docker compose logs backend | grep "started"

Debe decir "Started AutoReserveBackendApplication" sin errores previos.

Error 3: "Cannot Connect from React Frontend" - CORS o Network Issues

Síntomas:

- Frontend carga pero muestra errores en consola del navegador
- Errores tipo: "CORS policy: Response to preflight request doesn't pass access control check"
- Los vehículos no cargan en la página
- El login intenta conectar pero falla

Causas Posibles:

- CORS (Cross-Origin Resource Sharing) no configurado en Spring Boot
- Frontend intenta conectar a localhost:8080 cuando está en otro dominio
- El backend aún no está listo cuando el frontend intenta conectar
- Conflicto de puertos en Docker

Diagnóstico:

- # 1. Abrir Developer Tools del navegador (F12)
- # 2. Ir a pestaña "Network" y "Console"
- # 3. Buscar requests que fallen
- # 4. Ver en logs si hay "CORS" o "403 Forbidden"

Desde terminal:

`docker compose logs backend | grep -i cors`

`docker compose logs frontend | grep -i error`

Solución Paso a Paso:

1. Verificar que el archivo API Client en frontend apunta a URL correcta

En frontend/src/services/api.js o similar debe tener:

`const API_URL = process.env.REACT_APP_API_URL || 'http://localhost:8080'`

O en producción: `'http://<IP>:8080'`

2. Verificar que Spring Boot tiene CORS habilitado

En backend debe estar configurado:

`@Configuration`

`public class CorsConfig {`

`@Bean`

`public WebMvcConfigurer corsConfigurer() {`

`return new WebMvcConfigurer() {`

`@Override`

`public void addCorsMappings(CorsRegistry registry) {`

`registry.addMapping("/api/**")`

`.allowedOrigins("*")`

`.allowedMethods("GET", "POST", "PUT", "DELETE")`

```
#         .allowCredentials(false);  
#     }  
# };  
# }  
# }
```

3. Verificar que frontend se comunica al backend
docker compose logs frontend | tail -20

4. Testear directamente desde terminal
curl http://localhost:8080/api/cars

5. Si falla, reconstruir con CORS habilitado
docker compose down
docker compose up -d --build

Verificación de solución:

- Abrir navegador en Developer Tools
- Ir a pestaña Network
- Recargar página
- Debe haber requests a /api/cars con status 200 (éxito)

Error 4: "Port Already in Use" - Conflicto de Puertos

Síntomas:

- Error al ejecutar docker compose: Bind for 0.0.0.0:80 failed: port is already allocated
- Error: Address already in use
- No puede levantar uno o más servicios

Causas Posibles:

- Otro servicio/aplicación está usando los puertos 80, 8080 o 3306
- Contenedores anteriores de AutoReserve no se detuvieron correctamente
- Nginx u otro servidor web está corriendo en el puerto 80

Diagnóstico:

Verificar qué está usando el puerto 80

sudo lsof -i :80

Verificar qué está usando el puerto 8080

sudo lsof -i :8080

Verificar qué está usando el puerto 3306

sudo lsof -i :3306

Solución Paso a Paso:

Opción 1: Detener todos los contenedores de AutoReserve

docker compose down -v

Opción 2: Si hay otros servicios usando los puertos, pararlos

sudo systemctl stop nginx # Si Nginx está corriendo

sudo systemctl stop apache2 # Si Apache está corriendo

sudo systemctl stop mysql # Si MySQL está instalado en el sistema

Opción 3: Cambiar los puertos en el docker-compose.yml

Reemplazar:

ports:

- "80:80" # cambiar a "8000:80"

- "8080:8080" # cambiar a "8081:8080"

Opción 4: Matar procesos específicos (último recurso)

NO RECOMENDADO pero si es necesario:

sudo kill -9 \$(sudo lsof -t -i :80)

Finalmente: Levantar nuevamente

docker compose up -d --build

Verificación de solución:

docker compose ps

Todos deben estar "Up"

Error 5: "Authentication/JWT Token Issues" - Login No Funciona

Síntomas:

- Registro funciona pero login falla
- Error: 401 Unauthorized
- Token JWT no se guarda o expira inmediatamente
- "Invalid token" después de login exitoso

Causas Posibles:

- Secret key de JWT no configurada correctamente
- Token no se envía en headers correctamente
- Token expirado o con formato incorrecto
- Backend no está validando tokens correctamente
- CORS está bloqueando headers Authorization

Diagnóstico:

1. Ver logs de authentication

```
docker compose logs backend | grep -i "jwt\|token\|auth\|401"
```

2. Probar login directamente

```
curl -X POST http://localhost:8080/api/auth/login \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"email":"test@example.com","password":"password123"}'
```

3. Ver si token se devuelve

Solución Paso a Paso:

1. Verificar configuración de JWT en application.properties

```
cat backend/src/main/resources/application.properties | grep -i jwt
```

Debe tener:

jwt.secret=tu-clave-secreta-muy-larga-aqui

jwt.expiration=86400000 (24 horas en milisegundos)

2. Verificar que el frontend guarda el token correctamente

En localStorage o sessionStorage después del login:

localStorage.setItem('token', response.data.token)

3. Verificar que el frontend envía el token en requests

Cada request debe incluir:

headers: {

'Authorization': 'Bearer ' + token

}

4. Verificar que CORS permite el header Authorization

En Spring Boot CorsConfig debe estar:

registry.addMapping("/api/**")

.allowedHeaders("Authorization", "Content-Type")

5. Reconstruir

docker compose down

docker compose up -d --build

6. Probar nuevamente el login

Verificación de solución:

- En Developer Tools → Network → Login request
- Response debe incluir campo token
- En Console: localStorage.getItem('token') debe mostrar un token
- Headers de requests posteriores deben incluir Authorization: Bearer <token>

Error 6: "Image Build Fails" - Dockerfile Errors o Dependencias Faltantes

Síntomas:

- Error durante docker compose up -d --build
- Mensaje: failed to solve with frontend dockerfile.v0
- Errores de Maven o npm durante build
- ModuleNotFoundError o package.json not found

Causas Posibles:

- Dockerfile tiene sintaxis incorrecta
- Falta archivo .dockerignore
- Dependencias en pom.xml o package.json no se resuelven
- Cambios de código no committeados afectan el build
- Permisos insuficientes para leer archivos

Diagnóstico:

Ver error detallado

```
docker compose build --no-cache backend 2>&1 | tail -50
```

```
docker compose build --no-cache frontend 2>&1 | tail -50
```

Verificar Dockerfile existe

```
ls -la backend/Dockerfile
```

```
ls -la frontend/Dockerfile
```

Verificar dependencias

```
cat backend/pom.xml | head -20
```

```
cat frontend/package.json | head -20
```

Solución Paso a Paso:

1. Limpiar todo (incluyendo imágenes)

```
docker compose down -v
```

```
docker system prune -a --volumes -f
```

2. Reconstruir sin cache

`docker compose build --no-cache`

3. Si hay errores en Maven (backend):

- Verificar pom.xml tiene todas las dependencias

- Verificar que no hay conflictos de versiones

- Intentar compilar localmente primero

4. Si hay errores en npm (frontend):

- Verificar package.json

- Verificar package-lock.json no está corrupto

- Intentar npm install localmente primero

5. Si son permisos:

`sudo chmod -R 755 backend frontend`

6. Levantar de nuevo

`docker compose up -d --build`

Verificación de solución:

`docker compose ps`

Todos deben mostrar "Up" sin errores previos en logs

6.2 Checklist de Verificación Post-Despliegue

Después de desplegar, verifica lo siguiente:

Verificación	Comando	Resultado Esperado
Docker Compose Status	<code>docker compose ps</code>	Todos los servicios "Up"

Verificación	Comando	Resultado Esperado
Conectividad MySQL	<code>docker compose exec backend mysql -h mysql -u autoreserve -p1234autoreserve -e "SHOW TABLES;"</code>	Muestra lista de tablas
Frontend Carga	<code>curl http://localhost/</code>	HTML de la aplicación
Backend Responde	<code>curl http://localhost:8080/api/cars</code>	JSON con array de vehículos o [] si está vacío
Registro Funciona	Crear usuario en navegador	Redirige al login
Login Funciona	Login en navegador	Muestra dashboard del usuario
Base de Datos Persiste	Crear un vehículo, reiniciar MySQL, verificar	Datos siguen existiendo

7. Referencias Y Fuentes

Documentación Oficial

- **Spring Boot Documentation:** <https://spring.io/projects/spring-boot>
- **React Documentation:** <https://react.dev>
- **Docker Documentation:** <https://docs.docker.com/>
- **Docker Compose Reference:** <https://docs.docker.com/compose/compose-file/>
- **MySQL 8.0 Documentation:** <https://dev.mysql.com/doc/refman/8.0/en/>
- **Tailwind CSS Documentation:** <https://tailwindcss.com/docs>
- **JWT (JSON Web Tokens):** <https://jwt.io/>

Estándares y Normas Aplicadas

- **ISO/IEC 14764:2006:** Software Engineering – Maintenance of computer programs. Estándar internacional para mantenimiento de software aplicado en el desarrollo de este proyecto.

Marcos de Referencia

- **MINTIC - Guía para la Elaboración del Manual Técnico y de Operación de Sistemas de Información:** Guía G.SIS.01 del Departamento Nacional de Planeación (2020) que define el lineamiento LI.SIS.16 para documentación de sistemas.

Herramientas Utilizadas

- **GitHub:** <https://github.com/Leo-Quiroga/app-rentcar-sena>
- **DuckDNS:** <https://www.duckdns.org/>
- **AWS EC2:** <https://aws.amazon.com/ec2/>
- **PuTTY SSH Client:** <https://www.putty.org/>

Información del Proyecto

- **Nombre:** AutoReserve - Vehicle Rental Reservation Management System
- **Versión:** v1.0.0
- **Fecha de Lanzamiento:** 19 de junio de 2026
- **URL de Producción:** <http://autoreserve.duckdns.org/>
- **Repositorio GitHub:** <https://github.com/Leo-Quiroga/app-rentcar-sena>